

The Build and Publishing System

What `make` produces for the manuscript and for a typeset note, where the inputs come from, and
how a build becomes a review copy

Michael Volk

September 15, 2026

Abstract

Two families of document are typeset from the repository, and both are built by `make`. The manuscript in `paper/nature-biotech/` has its own Makefile and produces three views of one shared body: the journal submission, a drafting view with status chips and word budgets, and a published-like double column. Every typeset note under `notes-tex/` has a four-line Makefile that includes one shared file, `notes-tex/common/Makefile.common`, and produces the note’s PDF, a share view with the chips hidden, and a formatting, provenance and citation gate that builds nothing. Both families run the same engine, pin the same reproducibility variable so that identical sources give identical bytes, take their bibliographies from the same served store, and publish through the same script into a Zotero collection whose path is derived from the directory the files live in, with each build filed as a timestamped, content-hashed version. Neither Makefile ever produces a figure, a table or a number: those come from committed scripts in the experiment folders, and `make` only converts, assembles and checks.

Section status:  ready  author-reviewed, keep stable  not done, agents may edit freely

Contents

1	Two document families, one machinery 	2
2	The manuscript build 	2
3	A typeset note 	4
4	Where figures, tables and numbers come from 	5
5	Bibliographies 	6
6	Publishing a build 	6
7	Recipes 	7

1 Two document families, one machinery x

The **manuscript** is the Nature Biotechnology paper in `paper/nature-biotech/`. It uses the Springer Nature `sn-jnl` class with the Nature Portfolio style, keeps its body in one file that three thin wrappers include, and is the only document whose bibliography must come from the group library alone. A **typeset note** is any document under `notes-tex/<slug>/`: a design memo, a method review, an analysis report such as the additive-baselines documents. It uses the `article` class with the shared `tcdoc.sty`, which reproduces the manuscript’s drafting look, status chips and figure macros on top of a plain class that tolerates numbered sections and a contents page.

What they share. Both are compiled by Tectonic, a single binary with no TeX Live install, which fetches packages deterministically. Both export `SOURCE_DATE_EPOCH` as the newest modification time among the build inputs, so `hyperref` writes no wall-clock creation date and two builds of identical sources are byte-identical; that is what lets a review copy be versioned by content hash. Both take a committed `references.bib` that `make bib-pull` fetches from the served bibliography store, verified against a manifest. Both publish through `notes-tex/common/zotero_publish.py`, which derives the Zotero collection from the directory path. And both obey the same rule about content: no figure, table or number is authored inside a document; each is produced by a committed script under `experiments/<id>/` and only converted, placed or checked by `make`.

What differs.

Table 1: The two families side by side.

	Manuscript	Typeset note
Directory	<code>paper/nature-biotech/</code>	<code>notes-tex/<slug>/</code>
Makefile	its own, 250 lines	four lines, includes <code>common/Makefile.common</code>
Class	<code>sn-jnl</code> , Nature Portfolio style	<code>article</code> with <code>tcdoc.sty</code>
Body	<code>content.tex</code> , included by three wrappers	<code><slug>.tex</code> root, <code>sections/*.tex</code>
Default build	<code>make paper</code> , three PDFs	<code>make</code> , one PDF named for the directory
Figure sources	<code>draw.io</code> , exported headless	script-written SVG panels, converted; or <code>draw.io</code>
Figure gate	hard: any view refuses to build while a figure is out of spec	<code>make check</code> , after the build
Bibliography	group <code>paper</code> collection only	union of group and personal collections of the same name
Publish	<code>make publish</code> , <code>editing.pdf</code>	<code>zotero_publish.py <slug></code> , the default PDF
Collaborators	<code>sync-overleaf.sh</code> to the shared Overleaf clone	the Zotero review copy

Vocabulary. A **view** is one PDF rendering of the manuscript’s shared body. The **gate** is a check that reads a build’s log and sources and exits non-zero on a violation; the manuscript’s runs before Tectonic and blocks the build, the note’s runs after it and blocks nothing but the landing. A **status chip** is the colored mark after a heading, `todo`, `tent` or `final`, that appears in the contents so the contents page is the status board. A **provenance flag** is the visible marker `\external` or `\secondhand` on a fact not backed by the library mirror; it prints in every view. The **mirror** is the on-disk copy of the group’s papers under `$DATA_ROOT/torchcell-library/`, and `tc-lit` is the read-only service that streams it and the bibliographies to any machine with a key.

2 The manuscript build x

One body, three views. `content.tex` holds the manuscript. `submission.tex`, `editing.tex` and `twocolumn.tex` each include it and differ only in class options and in what the editing-only macros expand to: `\secstatus` draws a chip in the editing view and nothing in the other two, and the word-budget tags are counted only there. So `make paper` produces three PDFs from one set of edits, and `make editing` alone is the fast loop while writing.

Figures. Every manuscript figure is composed in `draw.io` at Nature print size and kept as an editable SVG under `notes/assets/drawio/NAME.drawio.svg`. `make figures` exports it headless to `figures/NAME.pdf`, the same base-name, so source and export are matched by name and nothing has to be listed. The set of figures to export is discovered from the `.tex`: any `figures/NAME.pdf` the body references that has a `draw.io` source on disk. A figure whose MathJax label rendered wider than its cell gets a crop-to-ink pass so the page lands at its true ink box without scaling the fonts. `check-figures.sh` then measures every PDF against the 180 by 240 mm print box and refuses any figure placed with a scaling macro; because every view depends on it, an out-of-spec figure cannot reach a PDF.

Word budgets. `wordcount.py` strips figure and table environments and the budget tags, counts the rest with `texcount`, and writes `wordcount.tex`, which the editing view embeds on its last page. `editing.pdf` depends on that file, so the embedded counts always match the compiled draft.

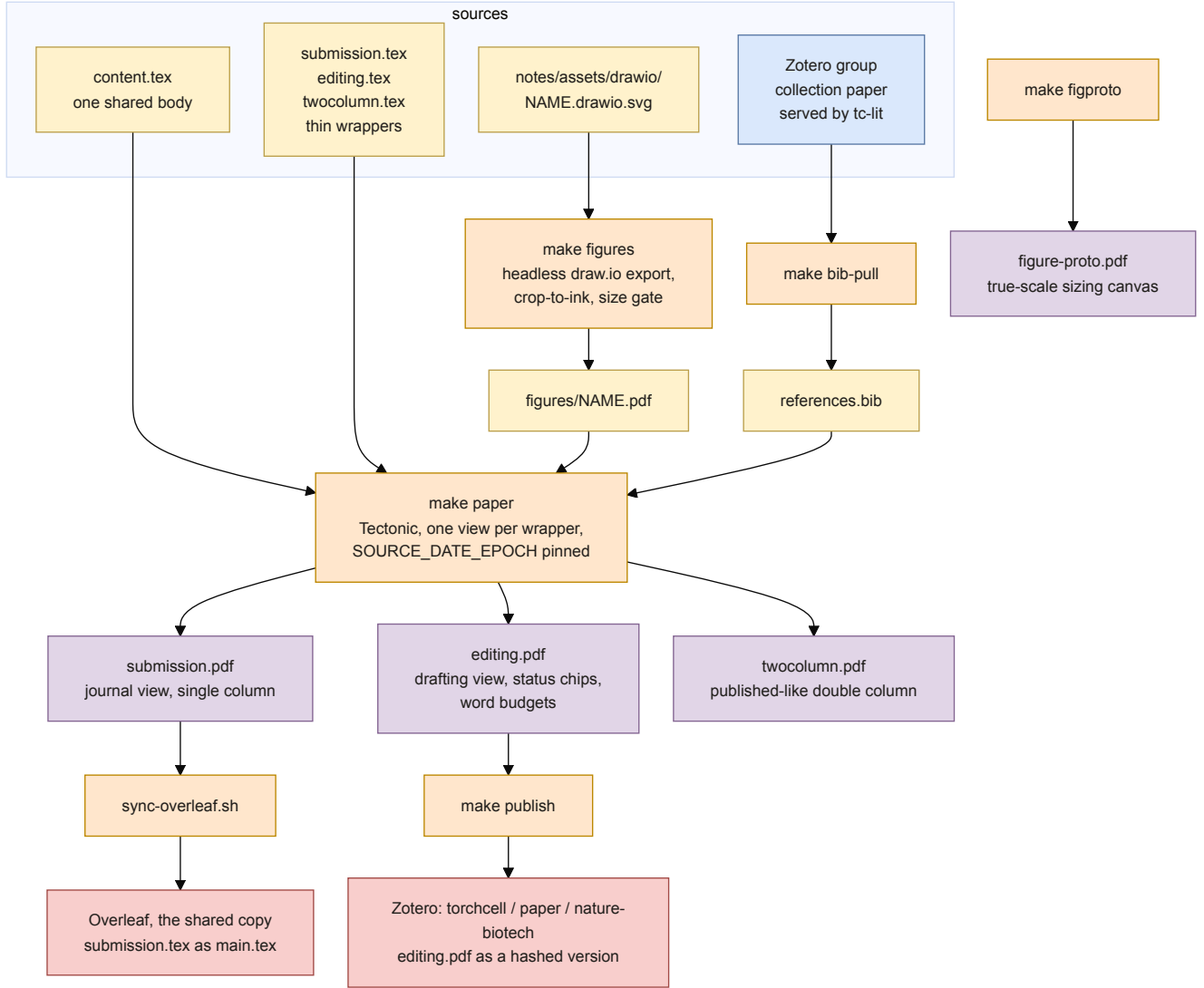


Figure 1: What `make` builds for the manuscript. Yellow boxes are source files, whether edited by hand or written by a script; orange boxes are make targets; purple boxes are built PDFs; blue boxes are outside the repository; red boxes are publish steps. One shared body reaches three views through three wrappers, and only the drafting view goes to Zotero.

Table 2: Manuscript targets. Every view depends on `checkfigs`, so a figure out of spec stops all three builds before Tectonic runs.

Target	Produces
<code>make paper</code>	<code>submission.pdf</code> , <code>editing.pdf</code> , <code>twocolumn.pdf</code> , then the word-count report
<code>make submission, editing, twocolumn</code>	one view
<code>make figproto</code>	<code>figure-proto.pdf</code> , the true-scale figure-sizing canvas
<code>make figlimits</code>	<code>figure-limits.pdf</code> , the Nature print-box card for collaborators
<code>make all</code>	paper, figproto and figlimits
<code>make figures</code>	re-exports every referenced figure whose draw.io source is newer than its PDF
<code>make fig</code>	re-exports every referenced figure regardless of timestamps
<code>make checkfigs</code>	the size and scale gate on <code>figures/*.pdf</code> , exit non-zero on a violation
<code>make wordcount</code>	<code>wordcount.tex</code> , the per-section counts against the budgets
<code>make bib-pull</code>	<code>references.bib</code> from the served <code>paper</code> bibliography
<code>make publish</code>	<code>editing.pdf</code> into Zotero; <code>publish-dry</code> , <code>publish-list</code> , <code>publish-submission</code>
<code>make flat</code>	<code>submission-flat.tex</code> , one file for the journal's upload

Two copies. The repository directory is the workshop: private, full, with the drafting scaffolding. `sync-overleaf.sh` copies a curated list of files into the Overleaf-backed clone that collaborators see, with `submission.tex` published as `main.tex`. It pulls first, so collaborators’ additions survive, and it propagates the workshop’s own deletions through a manifest without touching files it did not publish.

3 A typeset note x

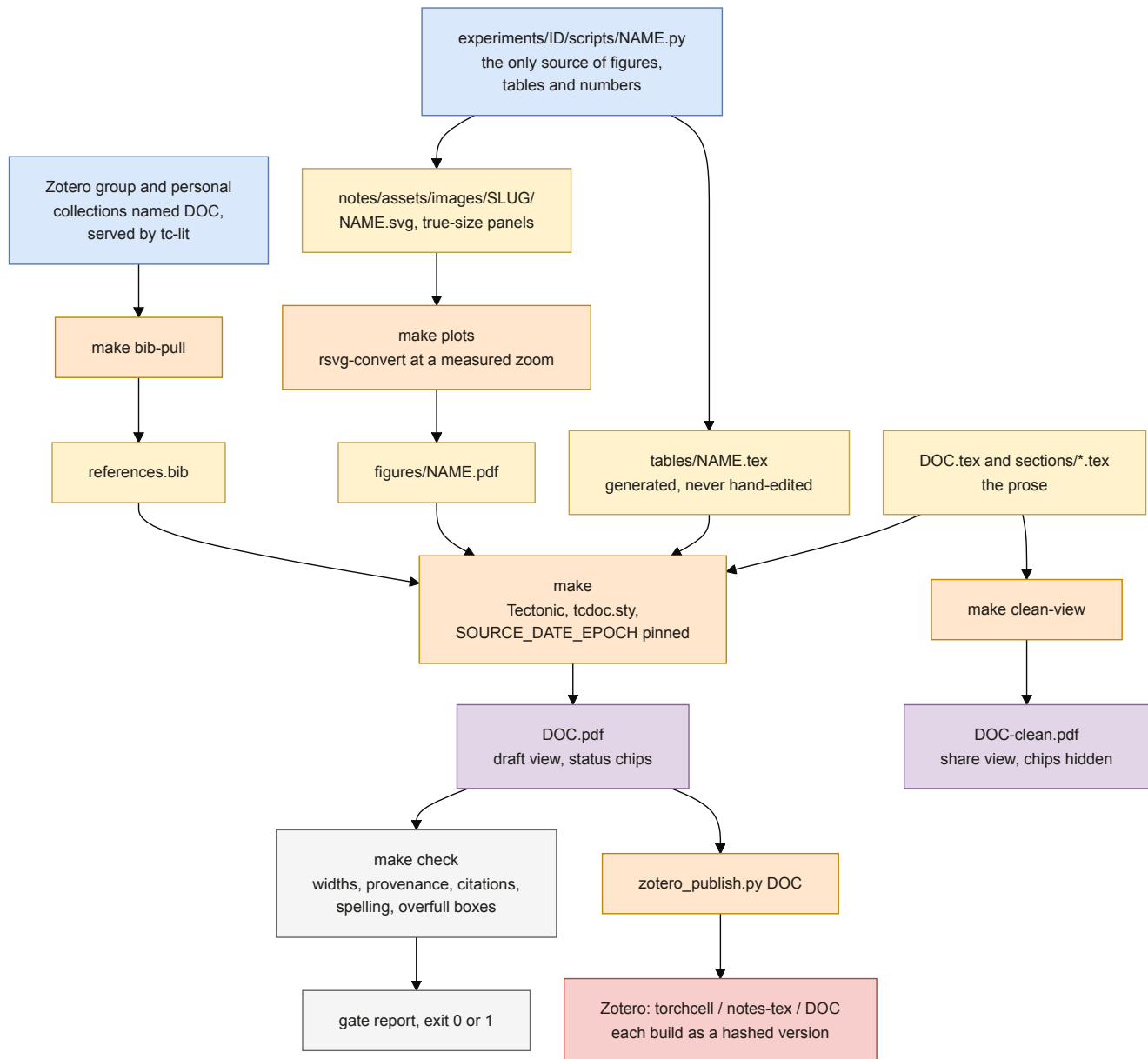


Figure 2: What `make` builds for a typeset note. Same colors as Figure 1; gray is the gate, which builds nothing. Figures and tables enter from the experiment scripts on the left; the prose is the only thing edited in the document directory.

Layout of a document directory. `<slug>.tex` is the root and is named for the directory, so the `eqtl-data-model` directory builds `eqtl-data-model.pdf`; four documents named `main.pdf` would be four files a file-opener cannot tell apart. `sections/*.tex` hold the prose, one file per top-level section. `tables/*.tex` are written by scripts and never edited by hand. `figures/*.pdf` are converted from script-written SVG panels or exported from draw.io. `references.bib` is pulled, not written. The Makefile sets at most a collection key and the image directories to read, then includes the common file. `tcdoc.sty` is symlinked into the directory at build time, because Tectonic does not honor `TEXINPUTS`; the link is ignored by git and any build recreates it.

The two views. The draft view carries the status chips in every heading, and so in the contents, which is the status board: `todo` may be edited freely, `tent` has been read by the author and is kept stable, `final` is ready. It also

Table 3: Targets every typeset note has, from the common Makefile, plus the two a document may add.

Target	Produces
<code>make</code>	<code><slug>.pdf</code> , the draft view with status chips and source pointers
<code>make clean-view</code>	<code><slug>-clean.pdf</code> , chips and pointers hidden, provenance flags kept
<code>make check</code>	nothing; the formatting, provenance, citation and style gate on the last build
<code>make plots</code>	<code>figures/NAME.pdf</code> from each referenced <code>notes/assets/images/<slug>/NAME.svg</code>
<code>make figures</code>	<code>figures/NAME.pdf</code> from each referenced draw.io source
<code>make bib-pull</code>	<code>references.bib</code> for this document from the served store
<code>make bib, bibcheck</code>	the same through Better BibTeX on a machine running Zotero; the collision report
<code>make watch</code>	rebuild on save
<code>make short</code>	<code><slug>-short.pdf</code> , a second root over the same sections (025-additive-baselines)
<code>make diagrams</code>	mermaid figures rendered and scaled to the text block (this document)

prints a gray source pointer under generated content. `make clean-view` rebuilds with the `clean` package option, which removes both, and keeps the provenance flags and their key, because an unverified number must not lose its flag when the document leaves the group.

Converting a panel. A plot script writes its panel as a true-size SVG whose width is in draw.io’s unit of 100 per inch, a number no renderer interprets the same way twice: librsvg has read it as points in one version and as CSS pixels in another, so one 179 mm panel converted to a 249 mm PDF on one machine and a 186 mm PDF on another. `make plots` therefore measures rather than assumes: it converts a 100-unit probe square, reads the size that came out, and converts every panel at the zoom that maps the probe to 72 points. The result is the panel at its true millimeters, placed with `\tcfig` at 1:1 so 6 pt type in the SVG is 6 pt on the page.

The gate. `make check` reads the last build’s log and every `.tex` file and reports six things: any heading without a status chip; any referenced figure wider than the 182 mm text block; any `\cite` key absent from `references.bib`; any file with a table but no `%% SOURCE:` line; British spelling and em-dashes in prose, with narration and self-reference as warnings; and overfull boxes above 5 pt from the log, which is how a table running off the page is caught without anyone reading page 8. Errors exit non-zero, warnings do not, since a `todo` section is allowed to be incomplete. Trailing restatement is not checked; an attempt ran at one true positive in ten and was removed.

A second build from the same sections. A document that needs a short version for sharing adds a second root that sets a switch, `\longfalse`, and wraps the long-only material in `\iflong ... \fi`. Its Makefile adds `make short` and `make check-short`; nothing in the common file changes, and the short PDF publishes with `-pdf <slug>-short`.

4 Where figures, tables and numbers come from ✗

The rule is that any artifact used in a document, a figure, a table, a derived number, is produced by a committed script in the experiment folder where the data were generated, reading the real result files. A document never contains a hand-drawn plot or a retyped value. That is what makes “where did this come from” answerable from the repository alone, and it is what `make` relies on: `make plots` converts what a script wrote, and nothing else.

Panels. A script under `experiments/<id>/scripts/` writes its panel to `$ASSET_IMAGES_DIR/<slug>/NAME.svg` and `NAME.png` through `savefig_true_size_svg`, at one of the standard Nature widths, Arial 6 pt, boxed axes, the ordered palette, panel letters at the outer top-left. The Dendron note paired with the script embeds the SVG. A document that reads panels from several experiments lists their slugs in `PLOT_SLUGS`, and the first directory holding `NAME.svg` wins. The additive-baselines document reads from `025-solid-growth` and `010-kuzmin-tmi`.

Tables. The same script writes any table the document needs as a LaTeX fragment into the document’s `tables/` directory, with a `%% SOURCE:` header naming itself, and the section `\inputs` it. Column widths for free-text columns are set in the generating script, since a fragment is never edited by hand.

Numbers in prose. A number quoted in the text traces to a result file the script wrote, usually a summary JSON beside the tables. The section that quotes it carries a `%% SOURCE:` comment naming the script and the file, invisible in the PDF and present for whoever edits the file. When a number is not backed by the mirror or by a repository result, it carries a visible flag: `\external{why}` for a fact from outside the holdings, `\secondhand{source}` for a value taken from a review’s table rather than the primary paper. The flags render in every view.

Diagrams. A system diagram like the two in this document is not data, and the rule for it is the weaker one that applies to draw.io figures: the source is committed, the export is reproducible by a `make` target, and the caption’s source comment names both. Mermaid sources live as Dendron notes and render through `notes/assets/publish/`

`scripts/mermaid_pdf.sh`, which uses the global `mermaid-cli` so that KaTeX labels typeset; a multi-line label needs a triple backslash as its row separator, and plain text and math are not mixed on one line.

5 Bibliographies ✗

Two tiers. The manuscript's `references.bib` comes from the group's `paper` collection and nowhere else. That is the publication guarantee: every work the paper cites lives in a library the whole group holds, so citations stay recoverable after publication. A typeset note's `references.bib` is the union of the group and personal collections of the same name as the document, because technical notes cite more widely than the paper and much of that reading lands in the personal library first; on a clean overlap the group entry wins.

Two routes to the same file. `make bib` exports through Better BibTeX on a machine with Zotero desktop running, and `make bibcheck` reports the two collision cases first: one citekey naming two different works, which is fatal because some citation would point at the wrong paper, and one work under two citekeys, which only splits a reference in the PDF. `make bib-pull` is the headless route: a nightly job on the mirror host exports every declared bibliography over the Zotero Web API into the mirror, pins each file's sha256 in a manifest, and `tc-lit` serves them. The pull verifies the bytes against both the manifest and the response header and leaves the target untouched on a mismatch. The two routes agree on citekeys and format fields differently, so a switch between them shows as a diff.

The gate on citations. `make check` fails on any `\cite` key that `references.bib` does not contain. That is the mechanical form of the rule that a paper is added to the library before it is cited, and it is the check that would have caught a citation to a work nobody held.

Who may write to the library. Almost nobody, and no agent. The library is curated by hand. The one sanctioned write is `zotero_publish.py` uploading a document's own built PDF into that document's output collection. Papers that are retrieved go to the mirror through the literature subsystem, which reads Zotero and never writes to it. Adding a reference is a curation decision that needs an explicit instruction each time; noticing that one is missing is not that instruction. The reason is the loop above: `make bib` regenerates the file from the library and `make check` fails on any key the export no longer has, so both an unrequested addition and its later cleanup change what a document is permitted to cite.

6 Publishing a build ✗

Why Zotero. The point of a review copy is annotation. Zotero's reader keeps highlights and comments attached to a specific PDF attachment, so a comment always refers to the exact rendering it was made on. A note saying a number is wrong is meaningless unless the build it was made against can be identified, and a shared folder does not give that.

Where a build is filed. The collection path is derived from the repository path, never configured: `notes-tex/<slug>` publishes into `torchcell / notes-tex / <slug>`, and `paper/nature-biotech/` into `torchcell / paper / nature-biotech`. The parent collection is a flat index of every document of that kind and the leaf holds the document's versions; the item is filed in both, because Zotero does not show a sub-collection's items in its parent by default. The topic collections that hold other people's papers are a separate tree, and keeping the two apart is what stops a draft from being read as literature when a bibliography is rebuilt. Two collections once shared the name `microbe-perturb-seq`, one for publishing and one feeding `make bib`, and deriving the path from the directory is what made that safe.

How a version is named. Each publish creates one child attachment named

`<doc>_<YYYY-MM-DD-HH-MM-SS>_<sha256[:8]>.pdf`

The timestamp is what a person sorts by; the hash identifies the bytes and is the same identifier the repository's provenance rule uses everywhere else. A re-run of `make` with nothing edited produces the same hash and is skipped as a duplicate, while any real edit produces a new one. This rests on the reproducible build: without `SOURCE_DATE_EPOCH` two builds of identical sources differed, and every publish uploaded a fresh near-identical PDF. The git commit and branch the build came from are written into the parent item, so a reviewer can go from a comment to the exact source state.

The commands. For a note, `zotero_publish.py <slug>` with `-dry-run` to preview, `-list` to see the versions, `-clean` for the share view and `-pdf <stem>` for any other PDF the document builds. For the manuscript, `make publish`

wraps the same script with `-pdf editing` and `-tex sections/frontmatter.tex`, because the title is declared there and the stock sample file in the same directory carries a placeholder title that a search would also find. Credentials are the repository's `.env`.

Landing. A document is edited in a git worktree like any other change, and the commit carries the built PDF and the converted figures beside the sources, so the PDF on `main` is always the one its sources produce. The branch is opened as a pull request and handed to the merge queue, which rebases, fast-forwards, closes the pull request and removes the worktree; publishing to Zotero can happen before or after, since the version is identified by content, not by commit.

7 Recipes ✗

Start a typeset note. Create `notes-tex/<slug>/` with `<slug>.tex`, `sections/`, an empty `references.bib`, and a Makefile that sets `PYTHON`, any `ZOTERO_COLLECTION`, and `PLOT_SLUG` or `PLOT_SLUGS` if the panels come from a directory not named for the document, then includes `../common/Makefile.common`. Give the root the three-line header every source file carries, pair it with a Dendron note of the same slug, and put `\secstatus{todo}` inside every heading. `make`, then `make check`.

Add a panel. Write it from the experiment script with `savefig_true_size_svg` into `$ASSET_IMAGES_DIR/<slug>/`, reference `figures/NAME.pdf` in a section with a `%% SOURCE:` comment above it, place it with `\tcfig`, and run `make plots` before `make`. A panel wider than the text block fails `make check`; redraw it at a standard width rather than scaling it.

Add a table. Have the script write `tables/NAME.tex` with a `%% SOURCE:` header, `\input` it inside a `table` environment with a numbered caption above, and set free-text column widths in the script.

Cite. Add nothing to the library. If the work is already in a collection of the document's name, `make bib-pull` and `\citep`. If it is not, name it by DOI in prose and record the missing reference; adding it is a separate decision for the author.

Publish and land. `make`, `make check`, read the rendered pages once, then `zotero_publish.py <slug>`. Commit the sources with the built PDF and the converted figures, open the pull request, and enqueue the branch.

Build the manuscript. `make -C paper/nature-biotech paper`. If a figure changed, `make figures` first, or `make fig` to force every export; the size gate runs on every build regardless. `make publish` for the review copy; `sync-overleaf.sh` for collaborators.